

Exact Execution Conditioning for Boolean Rule Infilling

Research supplement: algorithm, guarantees and comparison boundaries

Gabriel Jota Lizardo

October 7, 2026

Abstract

We extend a finite-original-token grammar posterior with execution profiles for Boolean JsonLogic rules. Conditional samples satisfy declared input-output examples while retaining the frozen dLLM product weights. The implementation uses the established covering-product transform. We give a tight scoped comparison: computing all union-profile masses needs 3^m products in a non-negative bilinear plan, whereas a signed plan needs exactly 2^m . Lean checks the underlying finite rectangle-cover obstruction. This is an incremental reference implementation and a delimited mathematical argument, not a claim of priority for transforms, program synthesis or exact inference. Independent human review and practical deployment evaluation remain pending.

1 Operation and antecedents

Given a fixed token canvas, finite original-token supports D_i , rational weights q_i , and m Boolean records, let $\mathcal{V}$ be the grammar-valid completions preserving committed positions. Every completion consumes exactly the declared slots with absent EOS. For a rule y , define its execution profile $h(y) \subseteq [m]$ by the records on which it returns true. We compute

$$Z_T = \sum_{y \in \mathcal{V}: h(y)=T} \prod_i q_i(y_i)$$

and sample tokens with this normalized conditional law when $Z_T > 0$. Rows truncated to a lexical domain retain their original normalization; exactness is on that domain. Zero mass is distinct from resource refusal.

The concrete language consists of Boolean fields, binary **and/or** and unary **!**, serialized as recursive JsonLogic objects. Matching the examples is a useful guarantee for completing a software-consumed filter or visibility condition. It is not correctness on unseen records. The reference supports single-letter ASCII fields and 1–12 records; it is not the entire JsonLogic language. Its grammar is independent of examples and desired labels.

Tree automata for synthesis from examples, including Boolean formulas, are established: see Wang et al., Example 6.2 and Theorem 6.3 [3]. Exact weighted inference is likewise established, and FactorDLM can represent deterministic semantic constraints [2]. The transform below is the classical covering product, not a new subset-convolution algorithm [1].

2 Inference and exact sampling

Lift each Boolean node of the unambiguous token-path forest to an array $W[S]$, $S \subseteq [m]$. A variable has its record-determined profile; punctuation is scalar. Alternatives sum arrays. NOT complements the indices. OR combines arrays by

$$(U \star V)[T] = \sum_{A \cup B = T} U[A]V[B].$$

AND applies the same product to complemented arrays and complements its output. With $\zeta U[S] = \sum_{A \subseteq S} U[A]$, compute $U \star V = \zeta^{-1}((\zeta U)(\zeta V))$ in $O(m2^m)$ ring work [1, Section 2.5, equations (15)–(17)]. Use exact integers scaled by one common denominator per token position; intermediate subtraction introduces no floating-point cancellation error. For sparse children, when the product of positive-support sizes is at most 2^m , the reference enumerates only those pairs; otherwise it uses transforms. The sampler uses the analogous sparse alternative. Both evaluate the same sums and retain the $O(m2^m)$ bound. This input-dependent dispatch is not itself a fixed non-negative bilinear plan.

Theorem 1 (Execution posterior). *The root array, divided by the product of row denominators, equals $(Z_T)_T$. For $Z_T > 0$, the recursive conditional sampler returns y with probability $\prod_i q_i(y_i)/Z_T$ on $\mathcal{V} \cap h^{-1}(T)$.*

Proof. Induct on increasing forest spans. Children occupy disjoint byte paths; combining their unique profiles implements the operator semantics. Sum over disjoint alternatives, using the original LL(1) unambiguous grammar. Aliases remain separate events because each original token closes on its own edge. Every full path has the same row denominator. For this restricted grammar, there are no unit/epsilon source productions: forced binarization and terminal proxies only distribute punctuation and pass single Boolean attributes. AND/OR have two Boolean children before binarization; NOT complements its single Boolean child. This is not a claim about arbitrary attributed-grammar normalization.

For OR conditioned on T , restrict V to subsets of T and compute $s(R) = \sum_{B \supseteq R, B \subseteq T} V[B]$ with a superset zeta transform. Draw $A \subseteq T$ with weight $U[A]s(T \setminus A)$, then B with weight $V[B]$ among $A \cup B = T$. Their joint probability is $U[A]V[B]/(U \star V)[T]$. AND uses complements; NOT and punctuation are deterministic in the profile. Choosing forest alternatives by their requested profile masses telescopes these conditional probabilities to the stated law. $\square$

3 A tight comparison for the required join

We compare the same complete profile operation on arbitrary dense arrays, not different decoding objectives or a deliberately poor factor encoding. A non-negative bilinear plan with r products has

$$C_T = \sum_{j=1}^r w_{jT} \left(\sum_A u_{jA} U_A \right) \left(\sum_B v_{jB} V_B \right), \quad w_{jT}, u_{jA}, v_{jB} \geq 0.$$

We count cross-input bilinear products; linear work and constant coefficients are additional costs. Allowing signed coefficients defines the signed class.

Theorem 2 (Bilinear product counts). *For all union-profile outputs, the minimum non-negative product count is 3^m . The minimum signed product count is 2^m . Complementing indices gives the same result for intersection-profile outputs.*

Proof. Non-negativity prevents cancellation of a wrong monomial. Each useful component therefore defines a rectangle $L_j \times R_j \times O_j$ on which $A \cup B = T$ holds. A fixed pair (A, B) has a unique output, so a useful rectangle cannot share different output profiles.

For each T , take the $2^{|T|}$ witness pairs $(A, T \setminus A)$, $A \subseteq T$. If a rectangle contains two distinct pairs for the same T , it contains both cross pairs. Their unions must also equal T , requiring $A' \subseteq A$ and $A \subseteq A'$. Thus each rectangle covers at most one witness. There are $\sum_T 2^{|T|} = 3^m$ witnesses, one of three possibilities per coordinate: neither side, left only, or right only.

The lower bound is attained by the products

$$U[A] \sum_{B \subseteq T, B \supseteq T \setminus A} V[B], \quad T \subseteq [m], \quad A \subseteq T.$$

A competent recursive implementation also attains it: split arrays on a bit, $C_0 = U_0 \star V_0$ and $C_1 = U_1 \star (V_0 + V_1) + U_0 \star V_1$. It uses 3^m products and $2(3^m - 2^m)$ additions.

The established zeta/product/inversion algorithm uses 2^m bilinear products and $3m2^{m-1}$ additions/subtractions. For optimality, the output polynomials are independent: the monomial $U[T]V[\emptyset]$ occurs only in C_T . A plan with r product polynomials spans at most an r -dimensional output space, hence $r \geq 2^m$. $\square$

The non-negative comparator is optimal in its declared class. The result does not lower-bound every factor encoding, a single target mass, adaptive sparse algorithms or all inference. A classical solver incorporating the same transform shares the advantage. No general superiority over EPIC or FactorDLM is implied. Rejection from the same syntax-conditioned law is also exact for this task and may be cheaper; its expected trial count is $1/\Pr(h(Y) = T \mid Y \in \mathcal{V})$ when that probability is positive.

4 Costs, implementation and evidence boundaries

Let F denote the explicit forest size and S the token support size. Preparation, including bounded normalization and token-DAG/forest construction, is additional. Evaluation uses $O(S + Fm2^m)$ arithmetic work and $O(F2^m)$ entries. Recomputed term contributions and conditional pair joins give an $O(Fm2^m)$ upper bound per sample. The reference also performs the existing scalar posterior evaluation for compatibility and mass-partition checks; its $O(F + S)$ cost is included. The method remains exponential in record count.

If span masses have at most L bits, transform intermediates have $O(L + m)$ bits. Each join therefore costs $O(m2^m(L + m) + 2^m M(L + m))$ bit work, where M is integer multiplication cost; input rationalization and output costs are separate. A safe L follows from the bit lengths of integer row weights and the product of row support sizes bounding token paths. Fewer products does not guarantee lower wall time on every input. Memory/work caps refuse unresolved inputs cooperatively; there is no hard preemption of a single large integer multiplication.

The small reference lives in `scripts/exact_commit/semantic_json.py` and is separate from production decoders. Focused offline tests use an independent JSON-tree executor and enumerate token products and integer sampling decisions, including aliases, fixed positions and zero mass. `formal/MWPC/SemanticProfiles.lean` verifies distinct finite ternary witnesses, the 3^m rectangle-cover obstruction, and the lower bound directly for natural-coefficient bilinear plans with a positive common tensor scale. Clearing rational denominators, the real-coefficient support reduction, signed optimality, sampling law and source implementation remain written proofs or tested obligations; the whole software is not Lean-verified.

A frozen opt-in MDLM/JsonLogic integration protocol is provided separately. It is an illustrative application check, not a held-out external benchmark or proof of semantic generalization. Human mathematical and novelty review are still required before promoting this supplement to the article’s main scientific claim. Known transforms and weighted synthesis are credited; priority for the delimited result is not asserted.

References

- [1] Andreas Björklund, Thore Husfeldt, Petteri Kaski, and Mikko Koivisto. Fourier meets möbius: fast subset convolution, 2006.
- [2] Jianchang Su and Wei Zhang. Constraints are graphs, not chains: Exact decoding for diffusion language models. *arXiv preprint arXiv:2609.32900v1*, 2026.
- [3] Xinyu Wang, Isil Dillig, and Rishabh Singh. Synthesis of data completion scripts using finite tree automata, 2017.